Fundamental Data Structures, Algorithms and Complexity, and Computer Architecture

What are CSC211 and its partner CSC212 about?

- CSC211 concentrates on *problem solving* that leads to correct programs whose timing and memory requirements are well understood. Students *develop programs* to prove they understand the theory. They *time* them and try to isolate what is slow, and *speed up their code*. Students learn how to validate and *prove that their code is correct*. Students learn many *problem solving approaches*.
- CSC212 has two parts— (1) It continues the themes of problem solving and programming in a high level language and understanding the time and space complexity of algorithms. Students continue to *broaden their problem solving skills*.
  (2) Hardware is covered and we learn some assembler programming to understand clearly what the underlying hardware does when it computes a loop or executes a function recursively. Different styles of computer architecture are compared and students gain an understanding of the interaction between high-level and low-level programming and hardware.

Fundamental Data Structures, Algorithms and Complexity

- This course concentrates on *problem solving* that leads to correct efficient programs.
- We will often first try *formulating problems* in English or pseudo code and then proceed to *code* them in Python, Java, C++, or even C# or whatever language you like the most.
- Representing data on computers is done using binary digits—called bits. We learn how eight, sixteen or more bits can be used to represent individual characters, how 32 or 64 bits can be used to represent integers and how to represent floating point numbers, and how other data structures are stored and what their memory requirements are.
- We revisit logarithms and the summing of simple series for insights about numbers.
- We will revise some of the important ideas you should already have mastered. We learn new viewpoints about data structures we already know well, such as linear lists, queues, stacks, hash tables, binary search trees, priority queues.

Fundamental Data Structures

- We learn algorithms to store and retrieve data in dictionaries, trees.
- We study efficiency of algorithms in terms of their time and space complexity.
- Algorithms are implemented using Java or Python.
- You are, however, expected to *become an expert in Java programming*.
- An integral part of this course consists of *your programming effort*.
- Most practicals must be completed on the same day they are issued.
- Marks contributing to your semester mark are awarded for original work submitted. Unoriginal submitted work gets marked down.

How your 15 notional hours should be divided.

There are *three hours of practical programming* per week in the Sun lab. *Three lecture hours* and *three tutorial hours* need to be done. You need to spend at least *six notional hours on self study* per week for this course.
Your B.Sc. programme requires 1200 hours per year, 600 hours per semester, which boils down to 200 hours per semester module, i.e., about 15 hours per week.

Class notes for 2016—core study material

These slides form the core of COS211/212.[1] Study the notes thoroughly.

Prescribed book for 2016

Algorithms FOURTH EDITION, by Robert Sedgewick and Kevin Wayne, Addison-Wesley, 2011.

Books for reference in 2016

Data Structures and Algorithms in Java, 4th Edition, by Michael T. Goodrich and Roberto Tamassia, John Wiley and Sons. Inc, 2006. There is a 5th edition.

Introduction to Algorithms, 3rd edition, Thomas H.Cormen, Charles E. Leiserson, Ronald L Rivest, and Clifford Stein, MIT Press, 2009. This is a daunting book that scares most people off but is the best.

Data Structures and Algorithms in Java, 2nd edition, Adam Drozdek, Thomson Course Technology, 2005. Prescribed in 2008.

[1] The class notes can be found on the Sun systems in the directory `/export/home/notes/ds/` in the files `ds-display.pdf`—or `ds-printable.pdf`.

For *Java programming* we recommend:
Savitch: the book you used last year.

Algorithms FOURTH EDITION, by Robert Sedgewick and Kevin Wayne is packed with examples of objects and methods in Java.

Chapter 1 of *Algorithms* FOURTH EDITION, by Robert Sedgewick and Kevin Wayne is available free on the Internet.
It is essential to have your own copy of the book.

For *Python* there are many good, free books available on the Internet. Stick to Python 3 and learn the differences between 2 and 3.

Fundamental Data Structures

Important *prerequisites* for this course are
—Proficiency in both Java and Python is expected before you enter this course. This is not a course about programming in Java or Python.
—Objects in Java.
—Translate pseudocode into runnable code
—Logarithms, summation of series, differential and integral calculus, linear algebra. Maths I is a prerequisite to this course.

At the conclusion of this course the aims are that you are expected:
—to program all the algorithms in this course proficiently in Java or in pseudo code.
—to explain any algorithm in this course coherently.
—to be able to understand, and discuss the time and space efficiency of each algorithm in this course.
—to know the problem-solving paradigms such as
divide-and-conquer, dynamic programming, brute force, breadth-first, depth-first or top-down design, recursion versus iteration, inductive evolution of programs, etc.

Outline for Dynamic Programming

Many problems seem to be infeasible when approached incorrectly.

Definition of Fibonacci numbers $F_n = F_{n-1} + F_{n-2}$, $F_1 = 1$, $F_2 = 1$.

Calculating Fibonacci numbers using the obvious recursive definition leads to an impractical exponential solution that is *very easy to understand*.

In some problems just guessing the exponential solution is daunting.

When a recursive solution has exponential running time, it is becomes easy to identify parts that are repeatedly called.

This solution for Fibonacci numbers has a useful property, it is divisible into *overlapping subproblems*. Since the n-th Fibonacci is determined by its two immediate predecessors, one of them has to be called again, and it in turn calls subproblems that have already been called.

Memoizing, *caching* or *storing* these overlapping parts can lead to scenarios where the exponential behaviour is reduced to polynomial—or even linear—behaviour, but needs some polynomially sized storage space.

Overview of Dynamic Programming (DP)

When tackling a problem using DP it is essential to *identify subproblems*, which tend to *be related recursively*.

If the subproblems *do not overlap*, and the recursion splits the problem evenly, such as in *mergesort*, or *quicksort*, it does not lead to exponential time complexity—those problems are solved by *sub-quadratic divide-and-conquer methods*.

Use dynamic programming when subproblems overlap. *In dynamic programming a call that is repeated must never be recalculated.*

When a procedure call is done for the first time its *value must be saved* so that this *work is never repeated*, and the time complexity of the repeated call is reduced to a constant at the expense of some memory.

Many problems become feasible with dynamic programming, or memoization.

A Note on Recursion

Consider the recursive definition for Fibonacci numbers

$$F_n = F_{n-1} + F_{n-2},\ F_1 = 1,\ F_2 = 1.$$

The n-th Fibonacci number F_n is defined in terms of the previous two.

In turn each of these are defined in terms of their predecessors.

Eventually F_3 can be found as the sum of the two given values $F_2 + F_1$.

For any recursive function the arguments must be shrunk or altered progressively towards some base case for which the value of the function is readily produced.

Some Dynamic Programming Problems

1. *Fibonacci numbers*. $F_1 = 1, F_2 = 1, F_n = F_{n-1} + F_{n-2}$.
2. *Binomial numbers*. $\binom{n}{n} = \binom{n}{0} = 1, \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$.
3. *Catalan numbers*. $C_n = \frac{1}{n+1}\binom{2n}{n}$.
4. Given a set $\{e_1, e_2, \ldots e_n\}$ in how many ways can the sum S be formed exactly by adding any number of each of these elements.
5. Given a set $\{e_1, e_2, \ldots e_n\}$ give the *minimum number of these elements needed* to form the S exactly by adding any number of each of these elements.
6. Given two strings X and Y give their *longest common substring*.
7. Given two strings X and Y give their *longest common subsequence*.
8. minimum edit Given two strings X and Y find the distance to turn X into Y using only *insert*, *delete* and *replace*.
9. Given a string of letters, $S = c_1 c_2 \ldots c_N$, find the minimum number of letters to prepend to S to turn it into a palindrome. This turns out to be similar to building the partial match table needed for doing Knuth-Morris-Pratt pattern matching.

More Dynamic Programming Problems

10. Given a string of letters, $S = c_1 c_2 \ldots c_N$, find the minimum number of letters to insert in order to turn S into a palindrome.
11. 0–1 *Knapsack*. Select from a set. Item $i \in N$ is either in or out of the knapsack, K. It has a value v_i and weight w_i and can join the knapsack if the total weight is less than the constraint C, i.e., $\sum_{i \in K} w_i < C$.
12. *Integer Knapsack*. Select n_i objects i with value v_i and weight w_i to be put into the knapsack K with a constraint of $\sum_{i \in K} n_i w_i < C$.
13. Given N find the *number of ways to tile* a $3 \times N$ area with 1×2 tiles.
14. Get the maximum size square matrix with all ones inside a matrix of ones and zeros.
15. Give the best parenthesization for multiplying the matrix chain $A_0 \times A_1 \times A_1 \times \ldots A_{n-1}$, assuming pairwise compatibility.
16. If N is a set of non-negative integers, detect a subset of N exists, that sums to S.
17. Total *number of different binary search trees* with n nodes.

Still More Dynamic Programming Problems

18. Given a tree, colour as many nodes red as possible without colouring two adjacent nodes.
19. Count all possible distinct binary strings with no consecutive 1s.
20. Format a string into lines of blocked text.
21. *Dropping eggs*.
22. *Jumping game*. Given a list of non-negative integers **`L`**, starting from **`L[0]`** go to the final element **`L[-1]`** by jumping. A **`jump`**, i.e., the number added to the current index **`i`**, can be at most the value at **`L[i]`**. So if **`jump`**$\leq$**`L[i]`**, then the next value of **`i`** is **`i + jump`**. The aim is for **`i`** to reach the **`L[-1]`** in the minimum number of jumps.
23. *Rod cutting*. Given a rod of n centimetres and a set of prices $\{p_i\}_{i=1}^{n}$, for each integer length i, maximize $\sum_{i \in T}^{n} r_i p_i$, where T is the *set* of rods cut from the rod of size n and r_i is the number of cuts of length i, i.e., how must the rod be *cut for maximum value*?
24. Find the *longest increasing subsequence* in a list of integers.
25. *Minimum weight triangulation*.
26. Given n dice, *count number of ways* to get sum S.

Still More Dynamic Programming Problems

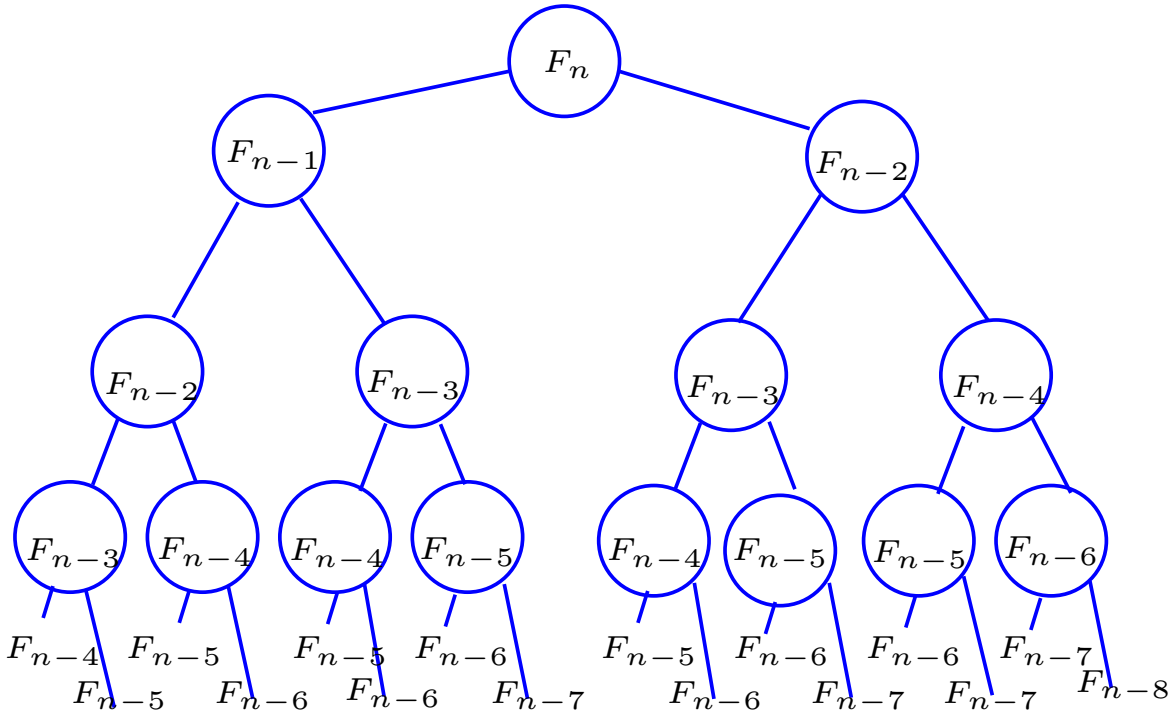
27. *Change maker*. Given a charge of A in rands and cents, and a tender of T in full Rands, give the number of different ways to give change using the 1c, 2c, 5c, 10c, 20c, 50c, R1, R2, R5, R10, R20, R50, R100, R200.
28. *Maximum contiguous subvector* can be done in $O(n)$ time with DP.
29. *Levenshtein distance* between two strings.
30. Many other DP problems.

Fibonacci Numbers—simple recursion

The first Fibonacci number $F_1 = 1$, and $F_2 = 1$, then $F_n = F_{n-1} + F_{n-2}$. The natural way to program Fibonacci numbers is to implement their definition directly.

```
1 def F(n):
2   if n <= 2:
3     return 1
4   else:
5     return F(n-1) + F(n-2)
```

This hack wastes time because each call starts up two new invocations, and often the function has already been evaluated many times for this argument.



k		Calls	$\Sigma_{r=0}^{k} F_{n-r}$
0	F_n	1	1
1	F_{n-1}	1	2
2	F_{n-2}	2	4
3	F_{n-3}	3	7
4	F_{n-4}	5	12
5	F_{n-5}	8	20
6	F_{n-6}	13	33
7	F_{n-7}	21	54
8	F_{n-8}	34	87

Fibonacci numbers—cached

k		Calls	$\Sigma_{r=0}^{k} F_{n-r}$
0	F_n	1	1
1	F_{n-1}	1	2
2	F_{n-2}	2	4
3	F_{n-3}	3	7
4	F_{n-4}	5	12
5	F_{n-5}	8	20
6	F_{n-6}	13	33
7	F_{n-7}	21	54
8	F_{n-8}	34	87

By inspection of the values it is easy to see that the total number of recursive calls is:

$$\text{Calls} = \Sigma_{r=1}^{n} F_r = F_{n+2} - 1.^{a}$$

By storing the value when it is calculated and recalling it on later invocations saves all the unnecessary work.

[a] Try to prove that it is true using induction.

```
1 F = {}
2 def fib(n):
3   if n not in F:
4     if n == 0 or n == 1:
5       F[n] = 1
6     else:
7       F[n] = fib(n - 1) + fib(n - 2)
8   return F[n]
9 print(fib(40))
```

This is called *memoization* or *caching*.

Binomial Coefficients

The number of combinations of k items out of a set of n items given by

$$C(n,k) = {}_nC_k = C_{n,k} = \binom{n}{k} = \frac{n!}{k!(n-k)!},$$

can be calculated using the recursive identity $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$, and programmed from the re-written form $C_{n,k} = C_{n-1,k} + C_{n-1,k-1}$, in Python as

```
1 def C(n,k):
2   if n == k or k == 0:
3     return 1
4   else:
5     return C(n-1,k) + C(n-1,k-1)
```

Binomial Coefficients Memoized

The number of combinations of k items out of a set of n items given by

$$C(n,k) = {}_nC_k = C_{n,k} = \binom{n}{k} = \frac{n!}{k!(n-k)!},$$

can be calculated using the recursive identity $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$, and programmed from the re-written form $C_{n,k} = C_{n-1,k} + C_{n-1,k-1}$, in Python as

```
1 cache = [[None for j in range(N+1)] for k in range(N+1)]
2 def c(n,k):
3   if cache[n][k] < 0:
4     if n == k or k == 0:
5       cache [n][k] = 1
6     else:
7       cache[n][k] = c(n-1,k) + c(n-1,k-1)
8   return cache[n][k]
```

Binomial Coefficients

If you only need few binomial coefficients then we use the recursive formulation

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \frac{n.(n-1)\ldots(n-k+1).(n-k)!}{k!(n-k)!} = \frac{n.(n-1)\ldots(n-k+1)}{1\,.\,2\,.\,3\ldots k}.$$

There are exactly k terms above the line. Do the calculation as follows:
$n/1$, $n/1 \times (n-1)/2$, $n/1 \times (n-1)/2 \times (n-3)/3, \ldots$.
Each division is always exact.

```
1 def C(n,k):
2   product = n
3   for r in range(2, k+1):
4     n -= 1
5     product = (product * n) // r
6   return product
```

Given any set S of N items, where item i has
a weight, $w_i > 0$, and has a benefit, $b_i > 0$,
such that the total benefit is

$$B = \sum_{i \in S} n_i b_i \text{ while } \sum_{i \in T} n_i w_i < C,$$

where C is the maximum weight that is called the constraint or the capacity of the container.
If the number of objects n_i is always one or zero, this is called the 0–1 knapsack problem and if n_i can take on any value it is called the integer knapsack problem.
Let T denote the subset of items selected then the objective is to

$$\text{maximize} \sum_{i \in T} n_i b_i \text{ while } \sum_{i \in T} n_i w_i < C.$$

The 0–1 Knapsack Problem

We will first consider the 0–1 knapsack problem.
Given any set S of N items where item i has
a weight, $w_i > 0$, and has a benefit, $b_i > 0$,
such that the total benefit is

$$B = \sum_{i \in S} b_i \text{ while } \sum_{i \in T} w_i < C,$$

where C is the maximum weight that is called the constraint or the capacity of the container.
If the number of objects is always one or zero, this is called the 0–1 knapsack problem.
Let T denote the subset of items selected then the objective is to

$$\text{maximize} \sum_{i \in T} b_i \text{ while } \sum_{i \in T} w_i < C.$$

The 0–1 knapsack problem: example

Given a set S of n items where item i has a weight, $w_i > 0$, and has a benefit, $b_i > 0$, then select some items that maximize the total benefit B, with weight of at most $C = 9$.

	1	2	3	4	5
weight	4	2	2	6	2
benefit	R20	R3	R6	R25	R80

The solution is
$T = \{S_1, S_3, S_5\}$, with weight $C = 8$.
By considering the possible solutions of this 0–1 knapsack problem as a binary number of five digits, it follows that there are 2^5 possible solutions. So, if, in general there are n items there are 2^n solutions.

0–1 Knapsack algorithm: Try I

Let $S = \{S_k\}_{k=1}^{n}$ be a set of items and let
$B[k] = \{\text{best selection from } S_k\}$.
This problem does not have *subproblem optimality*, for example the set

$$S = \{(3, 2), (5, 4), (8, 5), (4, 3), (10, 9)\}$$

of (benefit, weight) pairs with weight constrained by $C \leq 20$
Best selection from S_4:

$$B[4] = \{(3, 2), (5, 4), (8, 5), (4, 3)\}$$

Best selection from S_5:

$$B[5] = \{(3, 2), (5, 4), (8, 5), (10, 9)\}$$

Since we cannot derive $B[5]$ from $B[4]$ the problem does not possess *subproblem optimality*.

0–1 Knapsack algorithm: Try II

Let $S = \{S_k\}_{k=1}^n$ be a set of items and let

$$B[k, C] = \text{best selection from } S_k \text{ with weight at most } C.$$

This problem *has subproblem optimality.*

$$B[k, C] = \begin{cases} B[k-1, C] \text{ if } w_k > C \\ \max\{B[k-1, C], B[k-1, C-w_k] + b_k\} \end{cases}$$

i.e., the best subset of S_k with weight at most C is

either the best subset of S_{k-1} with weight at most C

or the best subset of S_{k-1} with weight at most $C - w_k$ plus item k.

0–1 Knapsack problem: algorithm

$$B[k, C] = \begin{cases} B[k-1, C] \text{ if } w_k > C \\ max\{B[k-1, C], B[k-1, C-w_k] + b_k\} \end{cases}$$

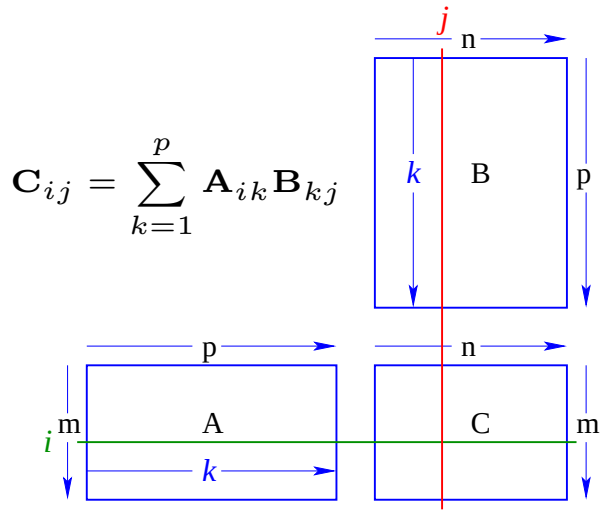
- Recall the definition of $B[k, C]$
- Since $B[k, C]$ is defined in terms of $[k-1, *]$, we can use two arrays of instead of a matrix
- Running time: $O(nC)$.
- Not a polynomial-time algorithm since C may be large
- This is a *pseudo-polynomial* time algorithm
  b_i—a positive "benefit"
  w_i—a positive "weight"

0–1 Knapsack problem: algorithm

Listing 1: 0–1 Knapsack problem: algorithm

```
1  Algorithm 01Knapsack(S, C):
2  In: set S of n items with benefit b_i
3      and weight wi; maximum weight C
4  Out: benefit of best subset of S with
5      weight at most C
6  let A and B be arrays of length C + 1
7  for w = 0 to C do
8     B[w] = 0
9  for k = 1 to n do
10   copy array B into array A
11   for w = w_k to C do
12     if A[w-w_k] + bk > A[w] then
13       B[w] = A[w-w_k] + b_k
14 return B[C]
```

Matrix multiplication



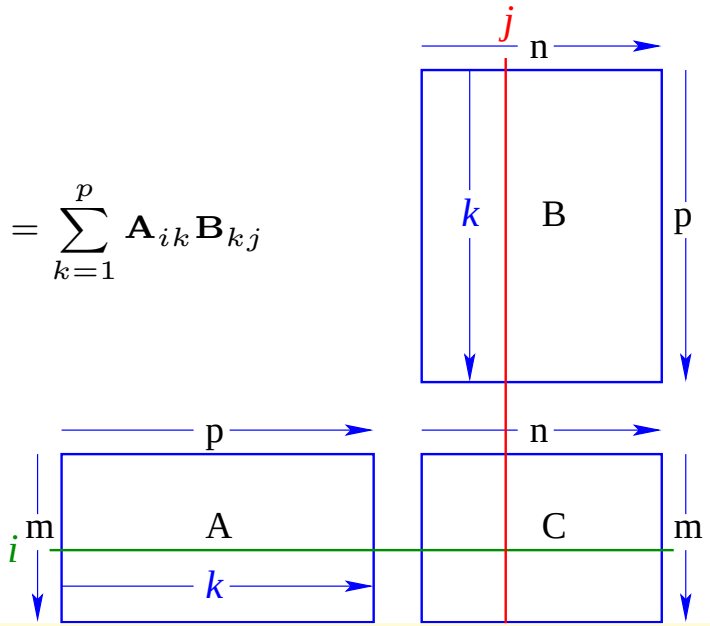
The ijth element of the $m \times n$ matrix $\mathbf{C}$, the product of the $m \times p$ matrix $\mathbf{A}$ and the $p \times n$ matrix $\mathbf{B}$ is:

$$\begin{aligned} \mathbf{C}_{ij} &= \mathbf{A}_{i1}\mathbf{B}_{1j} + \mathbf{A}_{i2}\mathbf{B}_{2j} + \ldots + \mathbf{A}_{ip}\mathbf{B}_{pj} \\ &= \sum_{k=1}^{p} \mathbf{A}_{ik}\mathbf{B}_{kj} \end{aligned}$$

$\mathbf{C}$ is calculated by running through all the possible combinations of i and j using two nested `for` loops.

Matrix multiplication

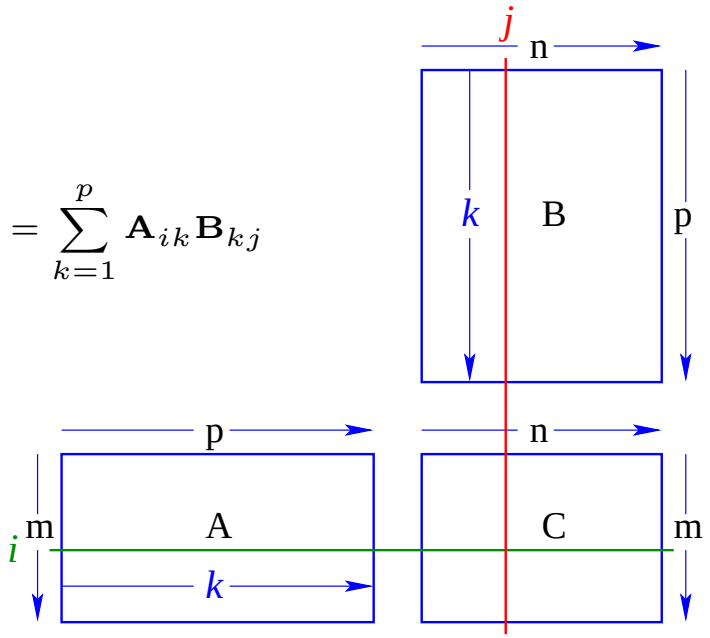
$$\mathbf{C}_{ij} = \sum_{k=1}^{p} \mathbf{A}_{ik}\mathbf{B}_{kj}$$




```
1  void matMultiply (double [][] A, double [][] B,
2                    double [][] C, int m, p, n) {
3  int i, k, j; double Cij;
4
5    for (i=1; i<=m; i++)
6      for (j=1; j<=n; j++){
7        Cij = A[i][1]*B[1][j];
8        for (k=2; k<=p; k++)
9          Cij += A[i][k]*B[k][j];
10       C[i][j] = Cij; } }
```

Matrix multiplication

$$\mathbf{C}_{ij} = \sum_{k=1}^{p} \mathbf{A}_{ik}\mathbf{B}_{kj}$$



Suppose $\{\mathbf{A}_i\}_{i=1}^{n}$ is a chain of compatible matrices with dimensions: $d_i \times d_{i+1}$.
We want to compute:

$$\mathbf{C} = \mathbf{A}_1 \times \mathbf{A}_2 \times \ldots \times \mathbf{A}_n$$

This is known as a chain product.
The dimension of the final product $\mathbf{C}$ is $d_1 \times d_{n+1}$.

Matrix Chain Products

Let $\{\mathbf{A}_\alpha\}_{\alpha=1}^{n}$ be a chain of compatible matrices with dimensions: $d_\alpha \times d_{\alpha+1}$.
We want to compute:

$$\mathbf{C} = \mathbf{A}_1 \times \mathbf{A}_2 \times \ldots \times \mathbf{A}_n$$

The products are done pairwise: e.g. $\mathbf{P} = \mathbf{A}_\alpha \times \mathbf{A}_{\alpha+1}$ where the elements of $\mathbf{P}$ are

$$\mathbf{P}_{ij} = \sum_{k=1}^{d_{\alpha+1}} \mathbf{A}_{\alpha ik} \mathbf{A}_{\alpha+1 kj}$$

The final dimensions of $\mathbf{P}$ are $d_\alpha \times d_{\alpha+2}$.
The problem is: Given the dimensions $\{d_\alpha\}$, how should the corresponding matrices be paired?

$\mathbf{B}$ is 3×100, $\mathbf{C}$ is 100×5, $\mathbf{D}$ is 5×5.
$(\mathbf{B} * \mathbf{C}) * \mathbf{D}$ takes $1500 + 75 = 1575$ ops
$\mathbf{B} * (\mathbf{C} * \mathbf{D})$ takes $1500 + 2500 = 4000$ ops

The Enumeration Approach is $O(4^n)$

Matrix Chain-Product Algorithm:

- Try all possible ways to parenthesize

$$\mathbf{A}_1 \times \mathbf{A}_2 \times \ldots \times \mathbf{A}_n$$

- Calculate number of operations for each one
- Pick the one that is best
- Running time:
- The number of parenthesizations is equal to the number of binary trees with n nodes
- This is exponential.
- This is the *Catalan* number, which is almost 4^n.
- This algorithm takes too long.

A Greedy Approach

- Idea α: repeatedly select the product that uses (up) the most operations.
- Counter example:

  $\mathbf{A}$ is 10×5, $\mathbf{B}$ is 5×10, $\mathbf{C}$ is 10×5, $\mathbf{D}$ is 5×10.

  Greedy idea α gives
  $(\mathbf{A} \times \mathbf{B}) \times (\mathbf{C} \times \mathbf{D})$,
  which takes
  $500 + 1000 + 500 = 2000$ operations.

  But the alternative bracketing yields:
  $\mathbf{A} \times ((\mathbf{B} \times \mathbf{C}) \times \mathbf{D})$ takes
  $500 + 250 + 250 = 1000$ operations.

Another Greedy Approach

- Idea β: repeatedly select the product that uses the fewest operations.
- Counter example

  $\mathbf{A}$ is 101×11
  $\mathbf{B}$ is 11×9
  $\mathbf{C}$ is 9×100
  $\mathbf{D}$ is 100×99
- Greedy idea β gives
  $\mathbf{A} \times ((\mathbf{B} \times \mathbf{C}) \times \mathbf{D}))$, which takes
  $109989 + 9900 + 108900 = 228789$ ops., but
  $(\mathbf{A} \times \mathbf{B}) \times (\mathbf{C} \times \mathbf{D})$ takes
  $9999 + 89991 + 89100 = 189090$ ops.
- The greedy approach β also does not yield the optimal value.

A Recursive Approach

Define subproblems: Find the *best* parenthesization of $\mathbf{A}_i \times \mathbf{A}_{i+1} \times \ldots \times \mathbf{A}_j$
Let $N_{i,j}$ denote the number of operations done by this subproblem.
The optimal solution for the whole problem is $N_{1,n}$.
Subproblem optimality: The optimal solution can be defined in terms of optimal subproblems.

- There has to be a final multiplication, i.e., the root of the expression tree, for the optimal solution. Suppose the final multiplication is at index i:

$$(\mathbf{A}_1 \times \ldots \times \mathbf{A}_i) \times (\mathbf{A}_{i+1} \ldots \times \mathbf{A}_n)$$

- Then the optimal solution $N_{1,n}$ is the sum of two optimal subproblems, $N_{1,i}$ and $N_{i+1,n}$ *plus* the time for the final multiplication.
- If the global optimum did not have these optimal subproblems, we could define an even better *optimal* solution.

A Characterizing Equation

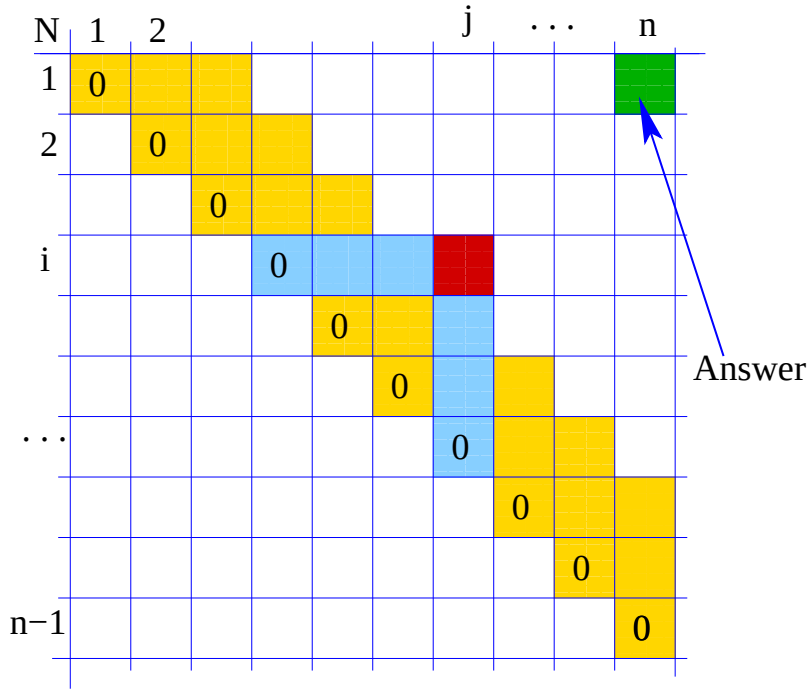
- Then the optimal solution $N_{1,n}$ is the sum of two optimal subproblems, $N_{1,i}$ and $N_{i+1,n}$ plus the time for the final multiplication.
- The global optimal has to be defined in terms of optimal subproblems, depending on where the final multiplication appears.
- Let us consider all possible places for the final multiplication: Recall that $\mathbf{A}_i$ is a $d_i \times d_{i+1}$ dimensional matrix. So, a characterizing equation for $N_{i,j}$ is the following:

$$N_{i,j} = \min_{i \leq k \leq j} \left[N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1} \right]$$

Note that subproblems are not independent—the subproblems overlap.

Visualizing dynamic programming

$$N_{i,j} = \min_{i \le k \le j} \left[N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1} \right]$$



First, *along the diagonal*, fill in the $N_{i,i} = 0$, $\forall i \in [1..n-1]$.
Get $N_{i,j}$ from previous entries in row i and column j.
Filling in each entry in the N table takes $O(n)$ time.
Actual parenthesization retrieved by recalling k for each N entry.
Total run time: $O(n^3)$. Space requirements n^2.

A Dynamic Programming Algorithm

Since subproblems overlap, we can't use recursion. Instead, we construct optimal subproblems *bottom-up*. Start with $N_{i,i} = 0$, $\forall i \in [1..n]$; then do length 2,3, ... subproblems. Running time: $O(n^3)$.

```
1  Algorithm matrixChain(S):
2  # pre: sequence S of n matrices to be multiplied
3  # post: number of operations in an optimal
4  # parenthesization of S
5
6  for i in range(1, n):
7    N[i][i] = 0
8  for b in range(1, n):
9    for i in range(n-b):
10     j = i+b
11     N[i][j] = +infinity
12     for k in range(1, j):
13       N[i][j] = min(N[i][j], N[i][k] +N[k+1][j] +
14                     d[i]*d[k+1]*d[j+1])
```

Dynamic programming in general

- Applies to a problem that appears to need a lot of time, possibly exponential, provided we have:
- *Simple subproblems*: the subproblems can be defined in terms of a few variables, such as j, k, l, m, and so on.
- *Subproblem optimality*: the global optimum value can be defined in terms of optimal subproblems
- *Subproblem overlap*: the subproblems are not independent, but instead they overlap: hence they should be constructed bottom-up.

Substrings

- A substring of a character string $x_0x_1x_2\ldots x_{i_{n-1}}$ is a string of the form $x_{i_0}x_{i_1}x_{i_2}\ldots x_{i_k}$, where $i_{j+1} = i_j + 1$ and $i_0 \geq 0$ and $i_k \leq i_{n-1}$.
- Example String: `"ABCDEFGHIJK"`
- Substring: `"ABCDEFGHIJK"`
- Substring: `"DEFGH"`
- Substring: `""`
- Not substring: `"DCEFGHIJ"`

Subsequences—Section 11.5.1 in Goodrich and Tamassia

- A subsequence of a character string $x_0x_1x_2\ldots x_{i_{n-1}}$ is a string of the form $x_{i_0}x_{i_1}x_{i_2}\ldots x_{i_k}$, where $i_j < i_j + 1$.
- Example String: `"ABCDEFGHIJIK"`
- Subsequence: `"ACEGJIK"`
- Subsequence: `"DFGHK"`
- Not subsequence: `"DAGH"`
- Subsequence, but also a substring: `"DEFGHI"`
- A subsequence is *not the same* as substring.

The Longest Common Subsequence (LCS)

- *Given two strings X and Y, the longest common subsequence (LCS) problem is to find a longest subsequence common to both X and Y*
- `"ABCDEFG"` and `"XZACKDFWGH"` have `"ACDFG"` as a longest common *subsequence*
- Note that a *subsequence* is not a *substring*:
  `"ZACK"` is not only a *substring* but it is also a *subsequence* of `"XZACKDFWGH"` a subsequence.
- Every *substring* of `T` is a *subsequence* of `T`.

The Longest Common Subsequence (LCS)—applications

- LCS has applications to DNA similarity testing—alphabet is
- LCS has applications to DNA similarity testing—alphabet is $\{\mathtt{A}, \mathtt{C}, \mathtt{G}, \mathtt{T}\}$
- LCS is used by spelling checkers to find the best word to fit your spelling mistake.
- LCS is used by `diff` in Linux/Unix to find differences between two files.

A Poor Approach to the LCS Problem

A Brute-force solution:

- Enumerate all subsequences of X
- Test which ones are also subsequences of Y
- Pick the longest one.

Analysis:

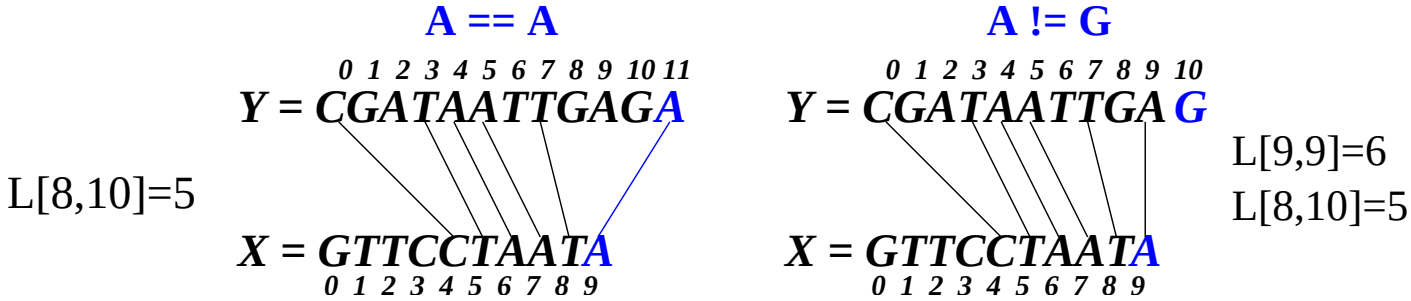
- If X is of length n, then it has 2^n subsequences
- This is an exponential-time algorithm

LCS solved with Dynamic Programming

- Define $L[i, j]$ to be the length of the longest common subsequence of $X[0..i]$ and $Y[0..j]$.
- Allow for -1 as an index, so $L[-1, k] = 0$ and $L[k, -1] = 0$, to indicate that the null part of X or Y has no match with the other.

Then we can define $L[i, j]$ in the general case as follows:

- `if` $x_i = y_j$, `then` \\ `add this match`
  $L[i, j] = L[i - 1, j - 1] + 1$
- `if` $x_i \neq y_j$, `then` \\ `no match here`
  $L[i, j] = \max\{L[i - 1, j], L[i, j - 1]\}$



An LCS Algorithm—in pseudocode

$$L_{i,j} = \begin{cases} \text{for } x_i = y_j & L_{i-1,j-1} + 1 \\ \text{otherwise} & \max\{L_{i-1,j}, L_{i,j-1}\} \end{cases}$$

L	−1	0	1	2	3			j	. . .	n−1	
−1	0	0	0	0	0	0	0	0	0	0	
0	0										
1	0										
2	0										
3	0										
	0										
i	0										
	0										
. . .	0										
n−1	0										

LCS Algorithm—closer to Java

Listing 2: LCS Java-like algorithm— $O(N^2)$

```
1  char [] void LCS(char X[], int n, char Y[], int m){
2  //pre: Strings X of length n, and Y of length m.
3  //post: For i in [1..n-1], j in [1..m-1]:
4  // L[i,j] = length of LCS of X[0..i] and Y[0..j]
5  int L[][] = new int [n][m];
6     for (i=1; i < n; i++)
7        L[i][0] = 0;
8     for (j=0; j < m; j++)
9        L[0][j] = 0;
10    for (i=1; i < n; i++)
11       for (j=1; j < m; j++)
12          if (x[i] == y[j])
13             L[i][j] = L[i-1][j-1] + 1;
14          else
15             L[i][j] = max(L[i-1][j], L[i][j-1]);
```

Listing 3: LCS—backtracking)

```
1  // Backtrack for longest common subsequence
2  int length = L[n-1][m-1];
3  char lcs[] = new char[length+1];
4    for (i = n-1, j = m-1; i > 0 && j > 0 && length > 0; )
5      if (x[i] == y[j]) {
6        lcs[length--] = x[i--];
7        j--; }
8      else if (L[i-1][j] > L[i][j-1])
9        i--;
10     else
11       j--;
12   return lcs; }
```

Analysis of LCS Algorithm

- The simple loops take $O(m)$ and $O(n)$ each.
- Outer nested loop takes $O(n)$ and repeats the inner loop $O)m)$ times.
- The body of the inner loop takes $O(1)$, i.e. it is independent of n or m.
- The backtracking is $O(1)$ per step and is at most $max(O(n), O(m))$, therefore it can be ignored.
- So, the total running time is $O(nm)$
- The result is in `L[n,m]` from which the subsequence may be recovered.

Divide-and-conquer methods

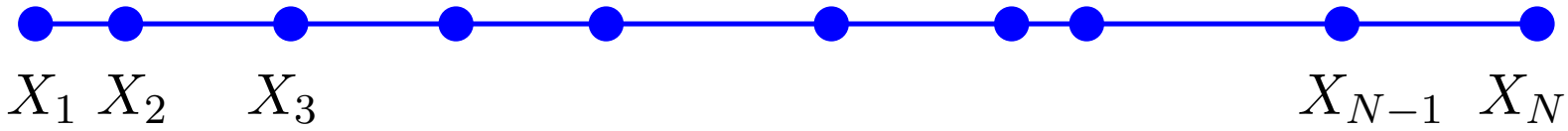
- In general, divide-and-conquer methods *divide* a problem with a set of N inputs X_i where $i \in [0..N)$ into k equal or almost equal parts.
- Each of these parts is then in turn divide-and-conquer-ed.
- Finally the results are combined.

```
1  def divide_and_conquer(X, low, high):
2    if high - low < small:
3      quickly resolve problem for small N in O(1) time
4      return result
5    else:
6      divide X into smaller parts P_1, P_2, ..., P_k, k ≥ 1
7      return combine(divide_and_conquer(P_1),
8                     divide_and_conquer(P_2),
9                     ...,
10                    divide_and_conquer(P_k))
```

- The resulting timing depends largely on the time taken by `combine`.
- If $T(combine) = O(N)$,
  $T(N) = kT(N/k) + O(N) = O(N \log_k N)$

An $O(N^2)$ method for 1-D closest pair

Given a set of N points in a straight line—use the X-axis for simplicity—find the closest pair of points.

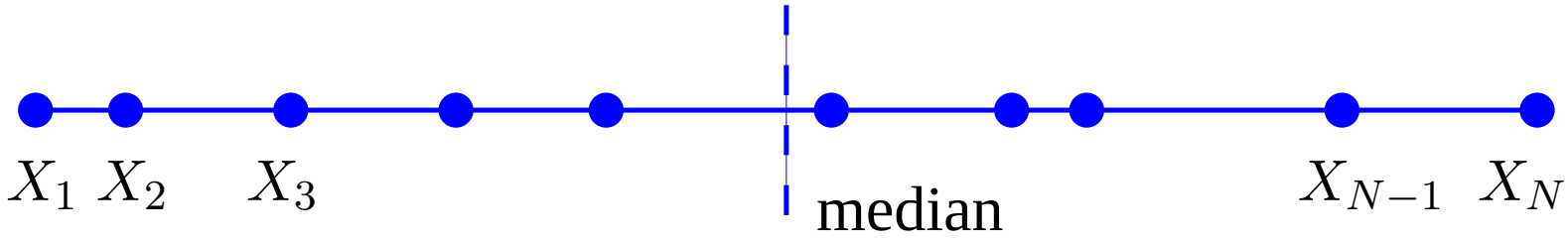


- The problem can be solved brute force by running through all the pairs in $O(N^2)$ time as follows

```
1   shortestDistance = |X[1] − X[2]|
2   indexJ = 1
3   indexK = 2
4   for j in [1..N-1]:
5     for k in [j+1..N]:
6       difference = |X[j] − X[k]|
7       if difference < shortestDistance:
8         shortestDistance = difference
9         indexJ = j
10        indexK = k
```

An $O(N \log N)$ divide-and-conquer method for 1-D closest pair

Given a set of N points on the X-axis for find the closest pair of points.



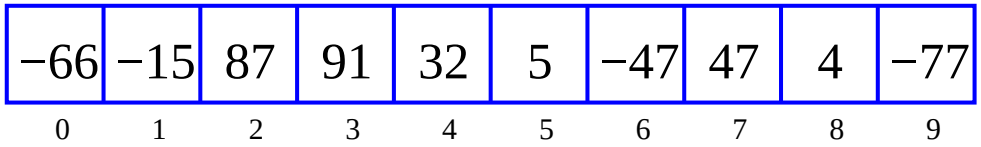
The problem can be solved in $O(N \log N)$ time using divide-and-conquer.

```
1  def closestPairs(X, low, high):
2    if high - low == 2: return |X[high] − X[low]|
3    elif high == low: return 1.79768e+308
4    else:
5       m = median(X, low, high)
6       L = closestPairs(X, low, m)
7       R = closestPairs(X, m+1, high)
8       δ =min(L, R)
9       straddle = minimum (X, m-δ, m+δ)
10   return min(L, R, straddle)
```

It takes time $T(N) = 2T(N/2) + O(N)$, i.e.,
$T(N) = O(N \log N)$.

The longest maximum subvector problem

In a list or vector X of N integers find the maximum sum of any contiguous subvector.[2]



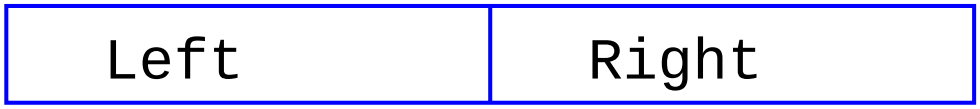
In the list X above the subvector $X[2..8]$ has the maximum sum 219.
Clearly if all the numbers are negative then the maximum contiguous subvector (MCS) is simply the largest number. In this case we define the MCS as 0.

We know how to solve this problem in $O(N)$ time but we are now interested in using divide-and-conquer to solve it in the slower time of $O(N \log N)$

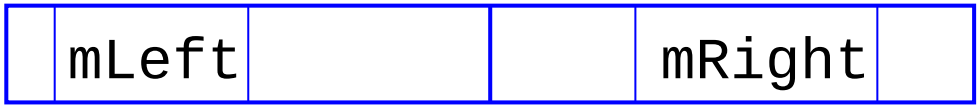
[2] This example comes from pages 77–81, "Programming Pearls", Jon Bentley, ACM Press and Addison-Wesley, New York, 2000.

A divide-and-conquer method for MCS

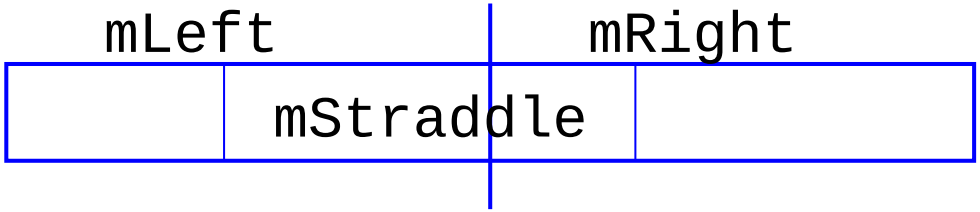
First, we divide the given array $X[\,]$ into two parts, `Left` and `Right`:



and then find the MCS `mLeft` in `Left` and the MCS `mRight` in `Right`.



There is still a snag—the MCS might straddle both `Left` and `Right`.



The MCS is the subvector that gives the biggest of these three values.
The final algorithm is (1) middle $= N/2$,
(2) Get MCS, `mLeft` of `X[low..middle]`, using
`mLeft` = `MCS(X, low, middle)`, and get the MCS, `mRight` of
`X[middle+1..N]`, using `mRight` = `MCS(X, middle+1, N)`.
More difficult is getting the straddling MCS, `mStraddle`.

Calculating the straddling MCS

The idea of the straddling MCS is first to calculate the MCS, called '`maxsofarLeft`' that ends at `X[middle]` but starts somewhere in `Left`—running backwards, and also calculating the MCS called '`maxsofarRight`' that starts at `X[middle+1]` and ends somewhere in `Right`. The sum of these two values is `mStraddle`.

```
1  int maxStraddle(low, high)
2     middle = (low + high) / 2
3     sum = 0; maxsofarLeft = 0
4     for i in [low..middle]
5        sum += X[middle - i + 1]
6        if sum > maxsofarLeft
7           maxsofarLeft = sum
8     sum = 0; maxsofarRight = 0
9     for i in [middle+1..high]
10       sum += X[i]
11       if sum > maxsofarRight
12          maxsofarRight = sum
13       return maxsofarLeft + maxsofarRight
```

A divide-and-conquer method for MCS

```
1  int MCS(low, high)
2     if low > high
3        return 0
4     if low == high
5        return max(0, X[low])
6     middle = (low + high) / 2
7     mLeft = MCS(X, low, middle)
8     mRight = MCS(X, middle+1, high)
9     mStraddle = maxStraddle (X, low, high)
10    return max (mLeft, mRight, mStraddle)
```

Suppose 'MCS' takes time $T(N)$ for an input of N. Then '`mLeft`' takes time $T(N/2)$ and '`mRight`' takes time $T(N/2)$. '`maxStraddle()`' takes $O(N)$ because, at worst, it scans the entire input once. We conclude that

$$\begin{aligned} T(N) &= 2T(N/2) + O(N) \\ &= O(N \log N) \end{aligned}$$

Divide-and-conquer methods—$O(N \log N)$

Suppose the divide-and-conquer method has time complexity $T(N)$ when it has an input of size N.
Divider-and-conquer methods entail:
(1) *dividing* the problem into equal parts,
(2) *solving each part* of the problem separately, and
(3) *joining* the results somehow.

1. *Dividing* the problem in two, has the low cost of $O(1)$. All that needs to be done is $N = N/2$.

2. *Solving each half* of the problem separately costs $2T(N/2)$.

3. *Joining the two results* is the critical part. Try to make this depend *linearly* on N. If it is sublinear all the better. Usually this part entails looking at every input once. In mergesort the two sorted halves are merged in time cN, making the cost of merge sort, $T(N) = 2T(N/2) + cN$.

4. In MCS straddle passes through all N elements once, giving the same formula.

Solving the recursive equation for merge sort and MCS

$$T(N) = 2T(N/2) + cN.$$

is quite easy.

$$\begin{aligned}
T(N) &= 2T(N/2) + cN \\
&= 2[2T(N/4) + cN/2] + cN \\
&= 2^2 T(N/2^2) + cN + cN \\
&= 2^2[2T(N/2^3) + cN/2^2] + cN + cN \\
&= 2^3 T(N/2^3) + cN + cN + cN \\
&= 2^3 T(N/2^3) + 3cN \\
&\vdots \\
&= 2^k T(N/2^k) + kcN
\end{aligned}$$

Continue until $N/2^k = 1$.

$$\begin{aligned} T(N) &= 2T(N/2) + cN \\ &\vdots \\ &= 2^k T(N/2^k) + kcN \end{aligned}$$

Continue until $N/2^k = 1$.
Since there is nothing to do when the input is of size 1, it follows that $T(1) = 0$, so we set $N/2^k = 1$ giving $N = 2^k$. This happens when $k = \log_2 N$ and

$$\begin{aligned} T(N) &= T(1) + kcN \\ &= kcN \\ &= cN \log_2 N \\ &= O(N \log N) \end{aligned}$$

An $O(n)$ method for MCS(X, low, high)

This method starts at X[low] and scans through all the elements up to X[high] keeping book of the maximum contiguous subvector found so far.

The maximum sum in the first i elements is either the maximum in the first $i-1$ elements, called 'maxSoFar' or that of the subvector that ends in X[i], called 'maxToHere'.
'maxToHere' contains the biggest sum *ending* at X[i − 1]. Considering item X[i] take the biggest of maxToHere + X[i] and 0.

```
1 int MCS(low, high)
2    maxSoFar = 0.0
3    maxToHere = 0.0
4    for i in [low, high]
5       maxToHere = max(maxToHere +X[i], 0.0)
6       maxSoFar = max(maxSoFar, maxToHere)
7    return maxSoFar
```